

บทที่ 5

การเขียนพีซีไอดีเวอร์สำหรับลินุกซ์

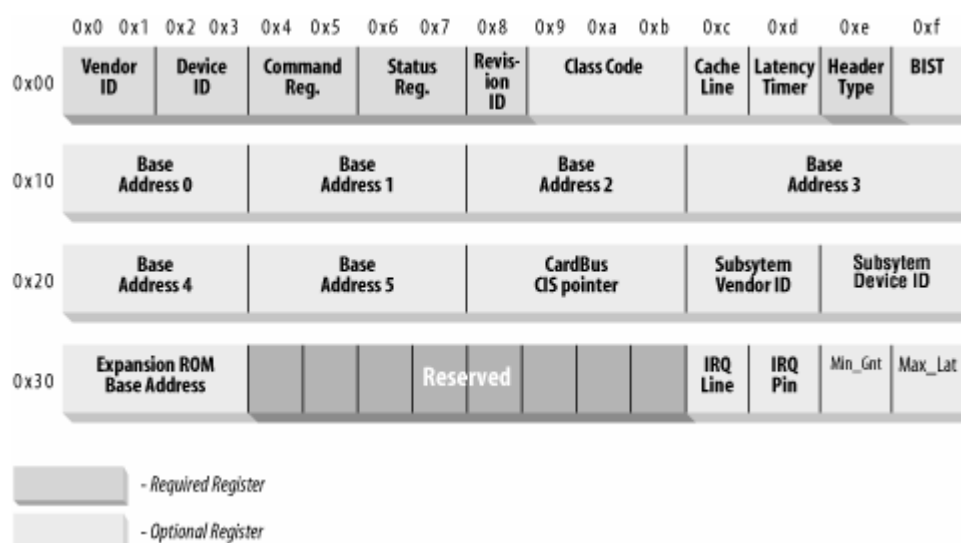
5.1 พีซีไอดีอินเตอร์เฟส

สถาปัตยกรรมพีซีไอนั้นได้สร้างมาเพื่อแทนที่สถาปัตยกรรมแบบไอเอสเอ มีจุดหมายหลักอยู่สามประการคือ 1. เพื่อเพิ่มประสิทธิภาพในการส่งข้อมูลระหว่างคอมพิวเตอร์กับอุปกรณ์ต่อพ่วง (peripheral) 2. เพื่อให้ไม่ขึ้นกับแพลตฟอร์ม (platform) ให้มากที่สุดที่จะเป็นไปได้ และ 3. เพื่อให้ง่ายในการใส่และถอดอุปกรณ์ต่าง ๆ ออกจากระบบ

พีซีไอบัสนั้นมีประสิทธิภาพสูงกว่าไอเอสเอเพราะใช้สัญญาณนาฬิกาที่มีความถี่สูงขึ้นกว่าสัญญาณนาฬิกานั้นจะทำงานอยู่ที่ความถี่ 25 หรือ 30 MHz (ขึ้นอยู่กับสัญญาณนาฬิกาของระบบด้วย) มีการส่งข้อมูลที่เป็นแบบ 32 บิต เรื่องของการไม่ขึ้นกับแพลตฟอร์มนั้นเป็นจุดมุ่งหมายสำคัญในการออกแบบคอมพิวเตอร์บัสและยังเป็นส่วนสำคัญที่เป็นคุณสมบัติของพีซีไอดี เพราะในโลกของพีซีไอดีสามารถนำพีซีไอดีไปใช้ในเครื่องที่เป็น IA-32, Alpha, PowerPC, SPARC64 และบางระบบอื่นได้เป็นอย่างดี

5.2 คอนฟิกูเรชันรีจิสเตอร์ (Configuration registers)

รูปด้านล่างนี้เป็นรูปที่แสดงตำแหน่งของรีจิสเตอร์ต่าง ๆ ของพีซีไอดีไวด์ ทุกพีซีไอดีไวด์ต้องมีพื้นที่อย่างน้อย 256 ไบต์ โดย 64 ไบต์แรกจะเป็นมาตรฐานที่เหลือก็แล้วแต่ดีไวด์แต่ละตัว



รูปที่ 5.1 มาตรฐานของ PCI Configuration registers

จากรูปจะเห็นว่ารีจิสเตอร์บางตัวนั้นจำเป็นต้องมี (required register) และบางตัวก็เป็นเพียงส่วนที่เสริมเข้ามา (optional register) ทุกพีซีไอดีไวัชจะต้องเก็บค่าที่มีความหมายชัดเจนในส่วนของรีจิสเตอร์ที่ต้องมี ส่วนเนื้อหาของรีจิสเตอร์ที่เป็นส่วนเพิ่มเติมนั้นขึ้นอยู่กับพีซีไวัชนั้น ๆ

สิ่งที่ต้องระลึกถึงไว้เสมอคือ พีซีไอร์จิสเตอร์นั้นจะเป็นลิทเติลเอนเดียน (little-endian) เสมอ ดังนั้นเรื่องของการเรียงลำดับของข้อมูลนั้นเป็นปัญหาที่ต้องระวัง

พีซีไอร์จิสเตอร์ที่ใช้ในการระบุไวัชแต่ละตัวมีอยู่สามตัวจากห้าตัว คือ เว้นเดอร์ไอดี (vendorID) ดีไวัชไอดี (deviceID) และคลาส (class) ผู้ผลิตพีซีไอดีไวัชจะกำหนดค่าที่เหมาะสมให้กับรีจิสเตอร์เหล่านี้โดยทำให้เป็นแบบอ่านได้อย่างเดียว ซึ่งผู้ที่เขียนไดรเวอร์สามารถใช้ข้อมูลตรงนี้ในการค้นหาไวัชได้ และในส่วนของ ซับซิสเต็มเว้นเดอร์ไอดี (subsystem vendorID) และ ซับซิสเต็มดีไวัชไอดี (subsystem deviceID) อาจจะถูกเซตเอาไว้ในบางครั้งโดยผู้ผลิตเพื่อใช้ในการแบ่งแยกความแตกต่างระหว่างไวัชที่เหมือนกัน

vendorID

เป็นรีจิสเตอร์ขนาด 16 บิต เป็นตัวบ่งบอกผู้ผลิตไวัชนั้น ตัวอย่างเช่น ดีไวัชของ intel จะมีเลขเว้นเดอร์ไอดีเป็น 0x8086 เป็นต้น

deviceID

เป็นรีจิสเตอร์ขนาด 16 บิต จะถูกเลือกโดยโรงงานที่ผลิต ไอดีตัวนี้จะใช้คู่กันกับเว้นเดอร์ไอดี กลายเป็นเลขที่ไม่ซ้ำขนาด 32 บิต ใช้ในการแยกแยะฮาร์ดแวร์แต่ละตัว มักจะใช้คำว่าลายมือชื่อ (signature) แทนคู่ของเว้นเดอร์ไอดีและดีไวัชไอดี

class

ทุก ๆ ดีไวัชจะมีคลาสอยู่ คลาสรีจิสเตอร์มีขนาด 16 บิต แบ่งออกเป็น 8 บิตบนเป็นเบสคลาส (base class) หรือกลุ่ม ตัวอย่างเช่น อีเทอร์เน็ต (ethernet) และ โทเค็นริง (token ring) ทั้งสองอย่างจะอยู่ในกลุ่มของเน็ตเวิร์ก (network) ขณะที่อนุกรม (serial) และขนาน (parallel) จะอยู่ในกลุ่มของการติดต่อสื่อสาร (communication) บางดีไวัชยังสามารถสนับสนุนหลาย ๆ อย่างบนดีไวัชเดียว

subsystem vendorID, subsystem deviceID

ส่วนนี้สามารถใช้ในการแยกแยะไวัชที่เป็นฮาร์ดแวร์ตัวเดียวกัน คือถ้าชิป (chip) ที่ใช้นั้นทำได้หลายอย่างบนโลคอลบัส แต่ทำงานต่างกันอย่างสิ้นเชิง และไดรเวอร์จะต้องแยกแยะแต่ละดีไวัชที่ทำงานด้วยได้

5.3 โครงสร้างของพีซีไอดีไวัช

พีซีไอดีไวัชนั้นมีอยู่สองแบบคือ แบบเก่าและแบบใหม่ โดยแบบใหม่จะยกเว้นเรื่องของการตรวจสอบดีไวัช (probing) และสนับสนุนการทำงานแบบฮอตปลั๊ก (hot plug) คือการที่สามารถ

ถอดและใส่อุปกรณ์ในขณะที่เครื่องยังเปิดอยู่ ส่วนแบบเก่านั้นการตรวจสอบดีไวซ์ผู้เขียนจะต้องทำเอง ดังนั้นถ้าเราไม่มีเหตุผลในการใช้งานการตรวจสอบแบบเก่าก็ไม่ควรใช้ หลังจากที่ไดรเวอร์นั้นพบดีไวซ์แล้วจำเป็นต้องทำตามขั้นตอนดังต่อไปนี้

เปิดการใช้งานดีไวซ์ (enable the device)

เข้าใช้งานพื้นที่ที่กำหนดไว้ในดีไวซ์ (access device configuration space)

ค้นหาทรัพยากรต่าง ๆ ที่ดีไวซ์จัดเตรียมไว้ให้ เช่น หมายเลขไออาร์คิว (IRQ numbers) และแอดเดรส เป็นต้น

จัดเตรียมทรัพยากรเหล่านั้น (allocate these resource)

ติดต่อกับดีไวซ์ (communicate with the device)

5.4 ไดรเวอร์แบบใหม่ (New-style)

ไดรเวอร์แบบใหม่นั้นมักจะเรียกฟังก์ชัน `pci_register_driver` ในตอนที่เริ่มต้นด้วยตัวชี้ (pointer) ไปยังสตรักเจอร์ (structure) ที่อธิบายไดรเวอร์ (`struct pci_driver`) โดยจะมีข้อมูลดังนี้

name ชื่อของไดรเวอร์

id_table ตัวชี้ไปยังตารางของดีไวซ์ไอดี

probe ตัวชี้ไปยังฟังก์ชันที่ใช้ในการตรวจสอบดีไวซ์ (probing function) จะถูกเรียกสำหรับทุกดีไวซ์ที่ตรงกับตารางไอดียังไม่ถูกดูแลโดยไดรเวอร์ใด ๆ ฟังก์ชันนี้จะส่งผ่านตัวชี้ที่ชี้ไปยัง `pci_dev` ที่เป็นสตรักเจอร์ที่ชี้แทนดีไวซ์และข้อมูลของตัวที่ตรงกันกับในตารางไอดี ถ้าไม่มีข้อผิดพลาดใด ๆ จะส่งค่ากลับเป็น 0 แต่ถ้าเป็นอย่างอื่นจะส่งค่ากลับเป็นหมายเลขที่ตรงกับข้อผิดพลาด (error code) ฟังก์ชันนี้จะถูกเรียกจากโพสเซสคอนเท็กซ์ (process context) เสมอ ดังนั้นจึงสามารถหลับ (sleep) ได้

remove ตัวชี้ไปยังฟังก์ชันที่ใช้ในการกำจัดค่าต่าง ๆ ซึ่งจะถูกรับในตอนที่ไดรเวอร์นั้นถูกถอดถอน (remove) ฟังก์ชันนี้จะถูกเรียกจากโพสเซสคอนเท็กซ์เสมอ ดังนั้นจึงสามารถหลับได้

save_state บันทึกลักษณะของดีไวซ์ก่อนที่จะถูกพัก (suspend)

suspend ทำให้ดีไวซ์อยู่สถานะพลังงานน้อย (low power state)

resume ทำให้ดีไวซ์กลับมาจากสถานะพลังงานน้อย

enable_wake เปิดให้ดีไวซ์สร้างสถานการณ์ปลุก (wake event) จาก สถานะพลังงานน้อย

ตารางไอดี นั้นเป็นอาร์เรย์ของ `struct pci_device_id` ที่จบด้วยศูนย์ทั้งหมด แต่ละส่วนประกอบด้วย

vendor, เว้นเดอร์และดีไวซ์ไอดีที่ตรงกัน (หรือใช้ `PCI_ANY_ID`)

device

subvendor, ซัพพลายเออร์และไดรเวอร์ไอดีที่ตรงกัน (หรือใช้ PCI_ANY_ID)

subdevice

class, คลาสของไดรเวอร์ที่ตรงกัน

class_mask คลาสแมสก์ เป็นตัวบ่งชี้ว่าใช้บิตไหนในการเปรียบเทียบ

driver_data ข้อมูลที่เป็นของไดรเวอร์นั้นเท่านั้น

ไดรเวอร์ส่วนมากจะไม่ต้องการใช้ส่วนของ driver_data

กรุณาใส่เครื่องหมายเหล่านี้ลงไปในส่วนของฟังก์ชันที่ใช้ตอนเริ่มต้นและส่วนของฟังก์ชันที่ใช้ตอนลบที่ควรจะวางนั้นมีหลักเกณฑ์ในการพิจารณาดังนี้

`__init` ไว้ในโค้ดที่จะถูกเรียกตอนเริ่มต้น และจะไม่ถูกใช้งานอีกหลังจากไดรเวอร์ผ่านส่วนของการเริ่มต้นไปแล้ว

`__exit` ไว้ในโค้ดที่จะถูกเรียกตอนออก ใช้ไม่ได้กับไดรเวอร์ที่ไม่ได้เป็นแบบโมดูล

`__devinit` ไว้ในโค้ดที่จะถูกเรียกตอนเริ่มต้นไดรเวอร์ เหมือนกับ `__init` ถ้าเคอร์เนลนั้นไม่ได้คอมไพล์ด้วย CONFIG_HOTPLUG

`__devexit` เหมือนกับ `__exit`

5.5 การค้นหาพีซีไอดีด้วยตนเอง (แบบเก่า)

พีซีไอดีไดรเวอร์ที่ไม่ได้ใช้ฟังก์ชัน `pci_register_driver()` ในการค้นหาพีซีไอดี จะค้นหาโดยใช้ขั้นตอนดังต่อไปนี้

การค้นหาโดยใช้เวเนเดอร์และไดรเวอร์ไอดี

```
struct pci_dev *dev = NULL;
```

```
while (dev = pci_find_device(VENDOR_ID, DEVICE_ID, dev))
```

```
    configure_device(dev);
```

การค้นหาโดยใช้คลาสไอดี

```
pci_find_class(CLASS_ID, dev)
```

การค้นหาโดยใช้ทั้งเวเนเดอร์/ไดรเวอร์ไอดี และซัพพลายเออร์/ไดรเวอร์ไอดี

```
pci_find_subsys(VENDOR_ID,    DEVICE_ID,    SUBSYS_VENDOR_ID,
```

```
SUBSYS_DEVICE_ID, dev)
```

ผู้เขียนไดรเวอร์สามารถใช้ `PCI_ANY_ID` แทนในส่วนของ `VENDOR_ID` และ `DEVICE_ID` ได้ ซึ่งจะหมายถึงเป็นไอดีทุกตัว ใช้สำหรับการค้นหาเฉพาะ `VENDOR_ID` หรือ `DEVICE_ID` นั้น ๆ

5.6 เปิดการใช้งานดีไวซ์ (Enable the device)

ก่อนที่จะเริ่มดำเนินการใด ๆ กับดีไวซ์ที่ค้นพบ จะมีการเรียกใช้งานฟังก์ชัน `pci_enable_device()` เสียก่อน เพื่อเป็นการเปิดการใช้งานพื้นที่ไอโอ (I/O region) และพื้นที่หน่วยความจำ (memory region) ของดีไวซ์ ของไออาร์คิวถ้าจำเป็น ปลุกดีไวซ์ถ้าดีไวซ์นั้นอยู่ในสถานะพัก

5.7 การเข้าถึงพื้นที่พีซีไอคอนฟิกสเปซ (PCI config space)

ผู้เขียนไดรเวอร์สามารถใช้ฟังก์ชัน `pci_(read|write)_config_(byte|word|dword)` ในการเข้าถึงคอนฟิกสเปซที่ไดไวซ์นั้นจัดเตรียมเอาไว้ผ่านทาง `struct pci_dev *` ทุกฟังก์ชันจะส่งค่ากลับเป็น 0 เมื่อทำงานสำเร็จหรือเป็นหมายเลขข้อผิดพลาด ที่สามารถแปลงเป็นข้อความได้โดยใช้ `pcibios_strerror` ไดรเวอร์ส่วนมากจะคิดว่าการเข้าถึงนั้นจะไม่ผิดพลาดบนดีไวซ์ที่ถูกต้อง

ถ้าไม่ใช่ `pci_dev` ก็สามารถใช้ผ่านทางฟังก์ชัน `pci_bus_(read|write)_config_(byte|word|dword)`

5.8 แอดเดรสและอินเทอร์รัพต์

แอดเดรสหน่วยความจำและพอร์ต และหมายเลขอินเทอร์รัพต์ ไม่ควรอ่านจากคอนฟิกสเปซ แต่ควรใช้ค่าที่อยู่ในสตรักเจอร์ `pci_dev` ซึ่งถูกเคอร์เนลทำการรีแมป (remap) ให้

ผู้เขียนไดรเวอร์ยังคงต้องใช้ `request_region()` สำหรับพื้นที่ไอโอ (I/O region) และ `request_mem_region()` สำหรับพื้นที่หน่วยความจำ เพื่อให้แน่ใจว่าไม่มีใครกำลังใช้งานดีไวซ์ตัวเดียวกัน

ทุกส่วนที่ดูแลอินเทอร์รัพต์ควรจะเป็นแบบ SA_SHIRQ และใช้ `dev_id` ในการแมปไออาร์คิวไปยังดีไวซ์